

Module 4 (Credit & Loans) - Implementation Learning Reference

This guide explains what the Module 4 code is doing in practical terms.

It is written for product, operations, risk, and compliance readers, not only backend developers.

It follows the actual implemented backlog in:

- ``backlogs/module-4-credit-loans-backlog.md``

And the implemented entry routes in:

- ``routes/api/v1/credit.php``

1) What Module 4 is responsible for

Module 4 owns the loan lifecycle:

1. Define loan products.
2. Create and review loan applications.
3. Run approval workflow (4 stages).
4. Generate repayment schedules.
5. Register collateral and guarantor obligations.
6. Disburse funds with accounting postings.
7. Record repayments and early settlements.
8. Assess arrears/penalties and track delinquency.
9. Support recovery, portfolio transfer, and credit reporting.

Important boundary:

- Module 4 orchestrates credit business logic.
- Authoritative accounting posting is integrated with Module 3 journal model.
- Cash-channel disbursement integrates with Module 5 teller/till controls.

2) How requests flow through the code

API controllers are thin:

- ``LoanController`` delegates to ``LoanWorkflowControllerAdapter``.

The adapter dispatches to specialized workflows:

- ``LoanCrudWorkflow`` (application CRUD)
- ``LoanSetupChargeWorkflow`` (setup charges, insurance premium collection)
- ``LoanApprovalWorkflow`` (step approvals and status transitions)
- ``LoanScheduleWorkflow`` (schedule generation and rescheduling)
- ``LoanRepaymentWorkflow`` (disbursement, repayment, arrears, early repayment)

This split is intentional: each workflow has a clear transaction boundary and testable business behavior.

3) Epic-by-epic implementation overview

Epic 1 - Loan Product Configuration

Core file:

- `app/Http/Controllers/Api/V1/LoanProductController.php`

What is implemented:

- Loan product CRUD with status lifecycle.
- Validation of ranges (amount, term, grace).
- Active ledger-account linking.
- Fail-closed formula-policy checks before accepting policy keys.

Why this matters:

- Product config errors can create systemic portfolio errors. The code blocks unsafe configs before activation.

Epic 2 - Loan Application & Setup

Core files:

- `app/Application/Loans/LoanCrudWorkflow.php`
- `app/Application/Loans/LoanSetupChargeWorkflow.php`
- `app/Application/Loans/AssessLoanSetupCharges.php`

What is implemented:

- Loan creation only for active + KYC-verified clients.
- Agency-scoped references for staff/accounts/sector data.
- Setup charge assessment (dossier fee, tax, guarantee deposit, insurance premium assessment where enabled).
- Direction-only manual exception decisions for dossier fee edge cases.
- Setup charge collection from customer account or teller cash channel, with journal entries.

Why this matters:

- Prevents “dirty origination” where the loan starts with invalid client scope or ungoverned setup fees.

Epic 3 - Four-Step Approval Workflow

Core files:

- `app/Application/Loans/LoanApprovalWorkflow.php`
- `app/Application/Loans/AdvanceLoanApproval.php`
- `app/Application/Loans/TransitionLoanStatus.php`

What is implemented:

- Required order: `montage -> comptabilite -> controle -> direction`.
- Step decisions: approved/rejected/returned.
- No skipping previous steps.
- Separation of duties: same approver cannot approve multiple distinct steps on same loan.
- Every status change recorded in `loan\_status\_transitions`.

Why this matters:

- Enforces real maker-checker governance with auditable step evidence.

Epic 4 - Amortization & Schedule Management

Core files:

- `app/Application/Loans/LoanScheduleWorkflow.php`
- `app/Application/Loans/GenerateLoanSchedule.php`
- `app/Application/Loans/RescheduleLoan.php`

What is implemented:

- Deterministic schedule snapshot creation (`loan\_schedule\_snapshots`, `loan\_schedule\_lines`).
- Formula-policy gates checked before generation.
- Versioned rescheduling while keeping history.
- Final-installment residual absorption so totals reconcile exactly.

Why this matters:

- Schedules are legal/operational contract artifacts. Deterministic generation + versioning protects traceability.

Epic 5 - Collateral & Guarantees

Core files:

- `app/Http/Controllers/Api/V1/CollateralController.php`
- `app/Http/Controllers/Api/V1/LoanGuaranteeObligationController.php`

What is implemented:

- Collateral lifecycle with agency controls.
- Collateral item sub-record lifecycle.
- Loan-specific guarantor obligations linked to verified guarantor records.
- Historical immutability controls so identity edits do not rewrite old obligation facts.

Why this matters:

- Preserves legal enforceability and post-facto audit truth.

Epic 6 - Disbursement & Accounting

Core files:

- `app/Application/Loans/LoanRepaymentWorkflow.php` (disburse endpoint workflow)
- `app/Application/Loans/DisburseLoan.php`

What is implemented:

- Only approved loans can disburse.
- Idempotent disbursement with replay mismatch protection.
- Transfer-account channel and teller cash channel.
- Teller/till preconditions for cash disbursement (open session, active till/ledger, sufficient cash).
- Journal entry creation and posting + disbursement record + status transition to `disbursed`.

Why this matters:

- Disbursement is the point where risk becomes money movement. Idempotency and precondition checks prevent double-loss and control breaches.

Epic 7 - Repayment, Early Settlement, Recovery

Core files:

- `app/Application/Loans/RecordLoanRepayment.php`
- `app/Application/Loans/EarlyRepayLoan.php`
- `app/Application/Loans/RecoverLoanFromAccounts.php`

What is implemented:

- Repayment allocation engine under policy gate (`repayment\_allocation\_order`).
- Schedule-driven allocation with component-specific posting (principal/interest/fees/insurance/tax/penalty).
- Overpayment retained on account (not silently consumed).
- Early repayment paths (default, interest waiver, negotiated-interest concession with direction governance).
- Automated recovery: recovery account first, then prioritized linked accounts.

Why this matters:

- Correct allocation is critical for customer fairness, portfolio quality, and regulatory reporting integrity.

Epic 8 - Arrears, Penalties, Delinquency, Transfer

Core files:

- `app/Application/Loans/AssessLoanArrearsAndPenalties.php`
- `app/Application/BatchRuns/ExecuteLoanArrearsAssessmentBatch.php`
- `app/Application/Loans/DelinquencyTrackingWorkflow.php`
- `app/Application/Loans/LoanTransferWorkflow.php`

What is implemented:

- Arrears and penalty assessment endpoint + batch execution.
- Formula-policy fail-closed behavior for arrears/penalty logic.
- Delinquency follow-up records with agency scoping.
- Loan transfer workflow with history preservation.

Why this matters:

- Turns portfolio servicing into repeatable operations, not manual one-off decisions.

Epic 9 - Credit Reporting

Core file:

- `app/Http/Controllers/Api/V1/ReportRunController.php`

What is implemented:

- Portfolio outstanding report.
- PAR delinquency report.
- Collection performance report.

Why this matters:

- These outputs are management and supervisory signals. They depend on the correctness of prior epics.

4) Key control patterns visible in the implementation

1. ****Fail-closed policy gates****

If formula policy is not approved, the workflow stops.

2. **Agency scope enforcement**

Most mutation paths reject cross-agency references.

3. **Idempotent money workflows**

Disbursement/repayment paths include idempotency and replay checks.

4. **Auditability first**

Security audit records are emitted for high-risk business actions.

5. **Separation of duties**

Approval stage controls prevent single-user rubber-stamping.

5) What “implemented” means here

Implementation is not just schema presence.

For Module 4, implemented means:

- routes/controllers/workflows exist,
- business constraints are enforced in code,
- accounting integration is wired for money events,
- and behavior is covered by dedicated tests.

Primary test suite:

- ``tests/Feature/Api/Module4CreditLoansTest.php``

Supporting unit checks:

- ``tests/Unit/Application/Loans/AssessLoanArrearsAndPenaltiesTest.php``

6) Practical reading order for non-backend stakeholders

If you want to understand and steer behavior quickly:

1. Read ``backlogs/module-4-credit-loans-backlog.md`` (business contract).
2. Read this guide (implementation narrative).
3. Read route map ``routes/api/v1/credit.php`` (what is exposed).
4. Validate one epic at a time through ``Module4CreditLoansTest.php`` test names.

This gives a clear bridge from business expectation to executable behavior.